BreastCare AI

Project Guide

Contents

BreastCare AI **1**
Contents . . . 1
What the app does . . . 1
Measurement fields . . . 2
Install and run . . . 2
Using the interface . . . 3
Model and evaluation . . . 3
Project files . . . 4
Data handling . . . 4
Limitations and next steps . . . 4
Troubleshooting . . . 4

BreastCare AI

BreastCare AI is a Python and Streamlit prototype for entering the 30 numerical measurements in the Wisconsin Diagnostic Breast Cancer (WDBC) dataset and viewing a Random Forest model output.

Current status: a trained Random Forest model is connected to the app. After all 30 measurements are entered, **Get prediction** displays the predicted dataset class and the model's class scores.

> **Clinical limitation:** This is an educational demonstration, not a diagnostic tool or medical device. The model was trained on a small public dataset and has not been validated for clinical use. Do not use the output to diagnose, treat, or make decisions about a patient.

Contents

- What the app does
- Measurement fields
- Install and run
- Using the interface
- Model and evaluation
- Project files
- Data handling
- Limitations and next steps
- Troubleshooting

What the app does

- Shows a prediction page with an overall 30-field progress bar.
- Groups inputs into **Mean**, **Standard error**, and **Worst** sections, each containing the same 10 measurements.
- Lets you switch sections using the large selectors above the measurement fields.
- Provides **Use test input** to fill all 30 fields with randomized example values.
- Provides **Clear inputs** to reset the current session.

- Sends complete inputs to the locally saved Random Forest model and displays its predicted WDBC class and class scores.
- Runs at `http://localhost:3001` by default.

The app does not include authentication, a patient directory, assessment history, or a database.

Measurement fields

Enter the numerical values from the source that generated the WDBC measurements. Each value must be non-negative. Avoid unit conversions unless they are required by the model's documented input schema.

Feature	Unit shown in the app
Radius	mm
Texture	gray scale
Perimeter	mm
Area	mm^2
Smoothness	ratio
Compactness	ratio
Concavity	ratio
Concave points	count
Symmetry	ratio
Fractal dimension	ratio

The sections represent the mean, standard error, and worst (largest or most irregular) values for each feature. The model bundle stores the expected input-key order, and the app uses that order to assemble the row passed to the estimator. The Random Forest was trained on raw values without scaling.

Install and run

Requirements

- Python 3.9 through 3.13
- pip
- A modern web browser

The app and training libraries are listed in `requirements.txt`. A compatible environment is already set up in `.venv` for this project.

Start the app

From the project directory, activate the environment and run Streamlit:

macOS/Linux:

```
source .venv/bin/activate
streamlit run app.py
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
streamlit run app.py
```

Open http://localhost:3001. The default host and port are set in `.streamlit/config.toml`. Stop the app with **Ctrl+C** in the terminal.

Create a new environment

If `.venv` is not present, install a supported Python version, then create the environment and install dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

On Windows, use `py -3.13 -m venv .venv` and activate it with `.venv\Scripts\Activate.ps1`.

Using the interface

1. Select **Mean**, **Standard error**, or **Worst**.
2. Enter the 10 values for that section. The selector and page progress display update as values are added.
3. Repeat until all 30 fields are filled.
4. Click **Get prediction** to display the predicted dataset class and class scores.
5. Click **Clear inputs** to reset the form.

Use test input fills the form with randomized values derived by perturbing an example WDBC feature vector. The values are for interface demonstration only; they are not independent test cases or clinical data.

The displayed class scores come from the Random Forest estimator. They are not calibrated estimates of an individual patient's cancer risk.

Model and evaluation

The app loads `models/breast_cancer_random_forest.joblib`. The bundle contains the fitted estimator, target labels, expected feature names and input keys, scikit-learn version, and holdout evaluation. `models/breast_cancer_random_forest.metrics.json` contains the detailed metrics from the training run.

The model was trained by `train_model.py` using scikit-learn's bundled WDBC dataset. No external CSV download is needed. The script uses a stratified 80/20 train/holdout split to report evaluation metrics, then fits the saved estimator on the full bundled dataset.

Holdout results from the included run:

Metric	Result
Accuracy	94.7%
Balanced accuracy	94.3%
Malignant sensitivity (recall)	92.9%
Benign specificity (recall)	95.8%
Malignant ROC AUC	0.994

The holdout contained 114 records. The confusion matrix, with rows as actual classes and columns as predicted classes in the order malignant then benign, was:

Actual \ predicted	Malignant	Benign
Malignant	39	3
Benign	3	69

These are results from one split of a small, public benchmark dataset. They do not demonstrate performance on new hospitals, populations, imaging workflows, or real patient cases and must not be interpreted as clinical validation.

Retrain the model

Activate `.venv`, then run:

```
python train_model.py
```

The script writes the model bundle and its metrics JSON to the `models/` directory, replacing the existing files. Optional parameters include:

```
python train_model.py --test-size 0.25 --random-state 7 --n-estimators 600
python train_model.py --output models/custom_model.joblib
```

The app expects the default model path and a compatible bundle schema. If you use a custom output path, copy it to the default path or update `MODEL_PATH` in `app.py`.

Project files

Path	Purpose
app.py	Streamlit interface, session-only form state, test input generation, input checks, and model inference.
train_model.py	Random Forest training, holdout evaluation, and artifact generation.
requirements.txt	Streamlit and machine-learning dependencies.
.streamlit/config.toml	Binds Streamlit to `localhost:3001`.
models/breast_cancer_random_forest.joblib	Model bundle loaded by the app.
models/breast_cancer_random_forest.metrics.json	Detailed holdout metrics from the included training run.

Data handling

The interface does not ask for a patient's name or medical record number. This app has no database and does not write entered measurements to files. Measurements are kept in Streamlit session state for the active session; clearing the form, ending the session, or stopping the app can discard them.

When deployed, entered values are sent to the computer or server running Streamlit so the app can perform inference. Hosting, proxy, and server logging are not configured by this repository. Avoid entering identifiable patient information.

Limitations and next steps

- The model uses a small, bundled public dataset and is intended for learning and interface demonstration.
- The held-out evaluation is one stratified split and is not an external or clinical validation.
- The app does not implement model calibration, clinical thresholds, uncertainty handling, or workflow review.
- The **Use test input** values are generated examples, not training data or valid clinical cases.
- Any real-world use would require appropriate representative data, independent evaluation, clinical oversight, and review of applicable regulatory requirements.

Troubleshooting

`streamlit: command not found`

Activate the virtual environment and install dependencies:

```
source .venv/bin/activate
python -m pip install -r requirements.txt
```

You can also start Streamlit as a module with `python -m streamlit run app.py`.

Port 3001 is already in use

Run the app on another port for this session:

```
streamlit run app.py --server.port 3002
```

The model cannot be loaded

Check that `models/breast_cancer_random_forest.joblib` exists and that the virtual environment dependencies are installed. Retrain it with `python train_model.py` if needed.

Test values or entered values disappear

Values are session-only. Use **Use test input** again or re-enter the values; the app has no save or recovery feature.